21 Parva Str., Gelemenovo 4444
BULGARIA
Tel. +359 889 244 444
Email: office@global-rts.com

ISO 9001* AQAP 2110*
ISO/IEC 22301
*GDPR *ISO 14001

Deployment & Operations Documentation

for the

EasyShare System

21 Parva Str., Gelemenovo 4444
BULGARIA
Tel. +359 889 244 444
Email: office@global-rts.com

ISO 9001* AQAP 2110*
ISO/IEC 22301
*GDPR *ISO 14001

Content

1. Overview

EasyShare is a web-based file management system built using Node.js and Express. It allows users to manage files and folders with user-based visibility permissions. Deployment involves configuring the backend server, PostgreSQL database, and environment variables securely.

2. Deployment Environments

Environment	Description	Domain / URL
Development	Local development	*http://localhost:3000*
Staging	Internal test server	*https://grtshare.bg*
Production	Live public deployment	*https://grtshare.bg*

Table 1: Deployment Environments

3. Dependencies

Ensure these are installed:

- Node.js (v16+)
- npm (v8+)
- PostgreSQL
- Multer (for file uploads)
- Express-session (for session management)
- CSRF protection middleware
- EJS (template rendering)
- Helmet for additional security headers
- Hpp for prevent HTTP parameter pollution
- Bcryptjs to hashing and dehashing
- Crypto to cripting and decripting bit string

- Redis and connect-redis to connect and configure redis stream control
- Joi to filtration user input
- Speakeasy and qrcode to generate secret and QR code for 2FA
- Dotenv to accept env variables
- Pg to connect to postgres database with pool

From *server.js*, these packages are used:

```
const express = require('express');

const bodyParser = require('body-parser');

const session = require('express-session');

const path = require('path');

const helmet = require('helmet');

const bcrypt = require('bcryptjs');

const crypto = require('crypto');

const multer = require('multer');

const hpp = require('hpp');

const Joi = require('joi');

const csurf = require('csurf');

const speakeasy = require('speakeasy');

const qrcode = require('qrcode');

const RedisStore = require('connect-redis').default;

const redis = require('redis');
```

Install using:

```
npm install express express-session pg csurf multer ejs bcryptjs dotenv helmet hpp joi nodemon pg qrcode speakeasy
```

Note: Before installing redis and connect-redis on your server, make sure that redis databases are installed and configured on your machine.

```
npm install redis connect-redis
```

4. Build & Deployment Steps

4.1. Clone the repository

```
git clone https://github.com/IChervenkov/EasyShare.git

cd EasyShare
```

4.2. Install packages

```
npm install
```

Set up environment variables (.env)

4.3. Start the server:

```
npm run dev
```

5. Server Configuration

- Configuration of multer for uploade files:

```
const upload = multer({

     storage: multer.memoryStorage(),

     limits: { fileSize: 100 * 1024 * 1024 }, // 100MB limit

});
```

- Configuration accept to the .env variable:

```
require('dotenv').config({
```

```
    override: true,

    path: path.join(__dirname, 'connect_db.env')

});
```

- Configuration Postgres database connection:

```
const { Pool, ClientBase } = require('pg');

const pool = new Pool({

    user: process.env.USER,

    host: process.env.HOST,

    database: process.env.DATABASE,

    password: process.env.PASSWORD,

    port: process.env.DATABASE_PORT

});
```

- Session Store and configuration:

```
this.app = express();

this.app.use(session({

    // store: new RedisStore({ client: redisClient }),

    secret: process.env.SESSION_SECRET || 'default_secret',

    resave: false,

    saveUninitialized: false,

    cookie: {

        secure: false,

        httpOnly: true,
```

```
            sameSite: 'strict',
            maxAge: 8 * 60 * 60 * 1000
        }
}));
```

Session Security Features:

- *httpOnly*: Prevents client-side JS from accessing session cookies.
- *sameSite*: *'strict'*: Mitigates CSRF risks from cross-origin requests.
- **Note**: secure: false should be true in production (with HTTPS).

- Redis configuration

```
const redisClient = redis.createClient({
    socket: {
        host: process.env.HOST_REDIS, // Provided by SuperHosting
        port: process.env.PORT_REDIS, // Default Redis port
        keepAlive: true
    }
});
redisClient.on('error', (err) => {
    // console.error('Redis Client Error:', err);
});
// Use Redis as the session store
(async () => {
    try {
```

```
            await redisClient.connect();

        } catch (err) {

            console.error('Connection Error:', err);

        }

})();
```

6. Logging & Monitoring

- Each user's actions are tracked back to their login on any device at any time
- Basic request logging can be added using morgan or custom middleware.
- Use external monitoring tools:
  - PM2 Monitoring
  - New Relic, Datadog, or UptimeRobot for uptime and performance
- Add crash handling and alerts for memory/storage errors

7. Backup & Restore

- **Database**: Use PostgreSQL tools:

```
pg_dump easyshare > backup.sql

psql easyshare < backup.sql
```

- **Uploads** (if not using memory storage): Back up /uploads folder.
- **Sessions**: Session table can be excluded from backup unless persistent session history is needed.

8. Operations Checklist

Task	Frequency	Notes
Test deployment on staging	Per release	Use production-like environment
Rotate session secrets	Quarterly	Update .env and restart server
Backup DB	Monthly	Automate with cron or cloud backup tools
Monitor memory usage	Continuous	Especially with memory-based multer config
Check file system usage	Weekly	Prevent disk full errors
Review security patches	Monthly	For Node.js and dependencies

Table 2: Operations checklist